



UTEC Posgrado



UTEC Posgrado

Una escuela para
Early Adopters

CURSO

INGENIERÍA DE DATOS

Mg. Angel Tintaya
Senior Data Engineer



The background of the image features a hand holding a glowing blue sphere. A network of white lines and dots, resembling a data or neural network, is visible in the upper left corner. The overall color scheme is dark blue with bright blue and white highlights.

DATAOPS Y AUTOMATIZACIÓN DE PROCESOS DE DATOS



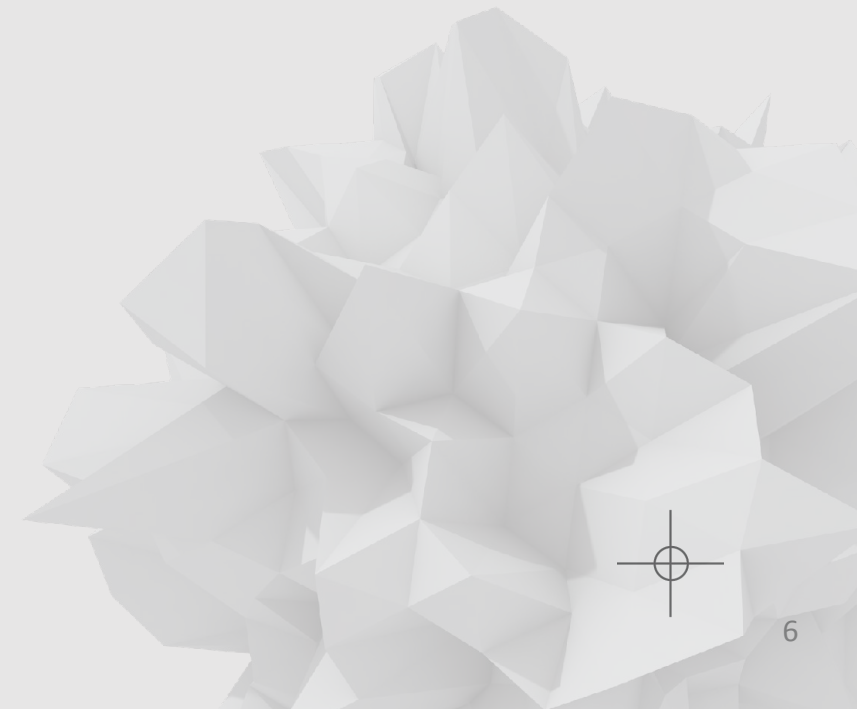
ORQUESTACIÓN DE FLUJO DE DATOS

¿Qué podría salir mal si un pipeline de datos falla... y nadie se da cuenta?



Qué aprenderemos hoy

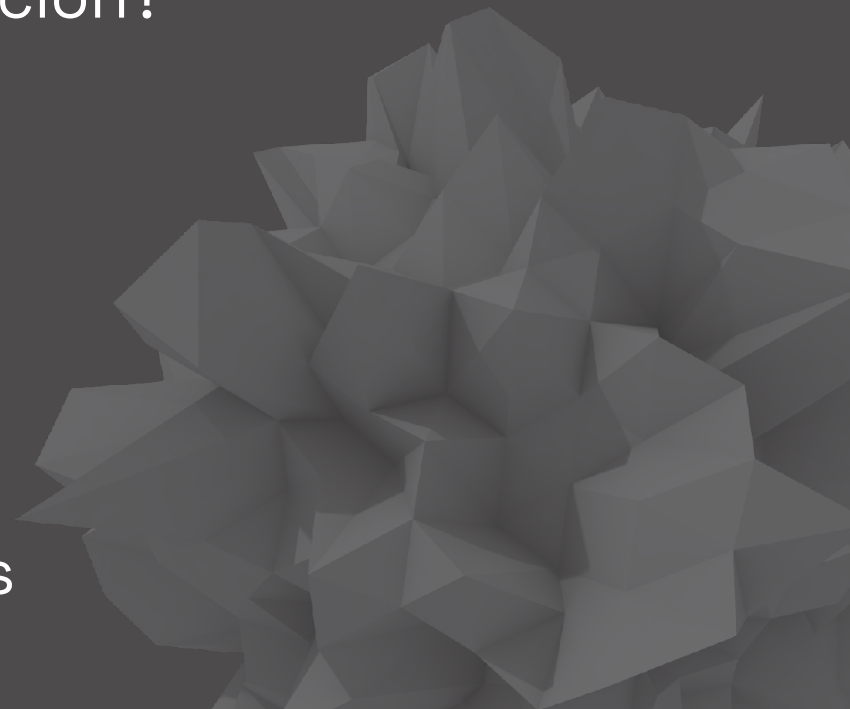
- Qué es la orquestación de datos
- Por qué los pipelines modernos necesitan coordinación
- Cómo funciona Apache Airflow
- Cómo funciona Databricks Workflows
- Cuándo usar cada enfoque
- Buenas prácticas para pipelines robustos





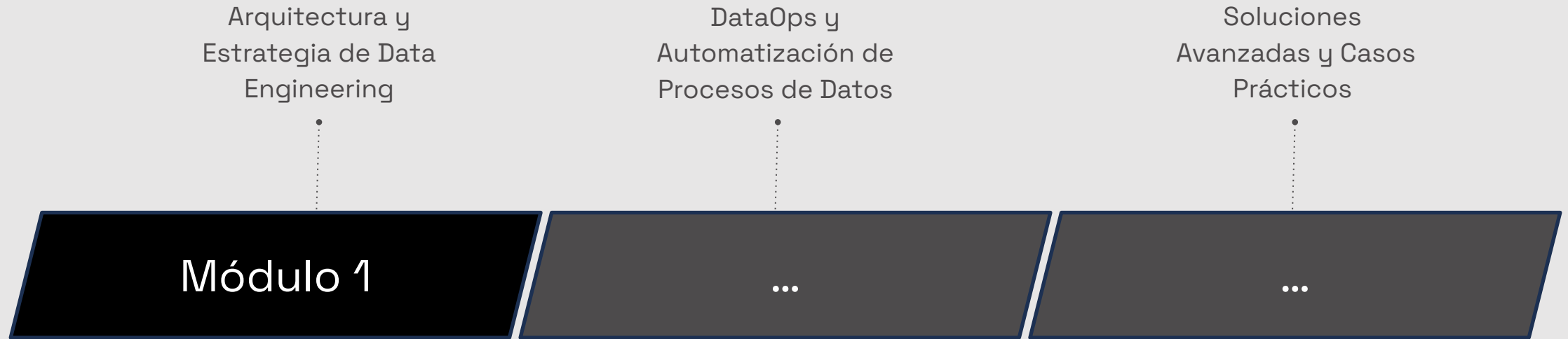
AGENDA DE HOY

- The road so far
- ¿Por qué los pipelines necesitan orquestación?
- Conceptos clave de orquestación
- Apache Airflow
- Databricks Workflows
- ¿Cuándo usar cada uno?
- Buenas prácticas para pipelines modernos





The road so far



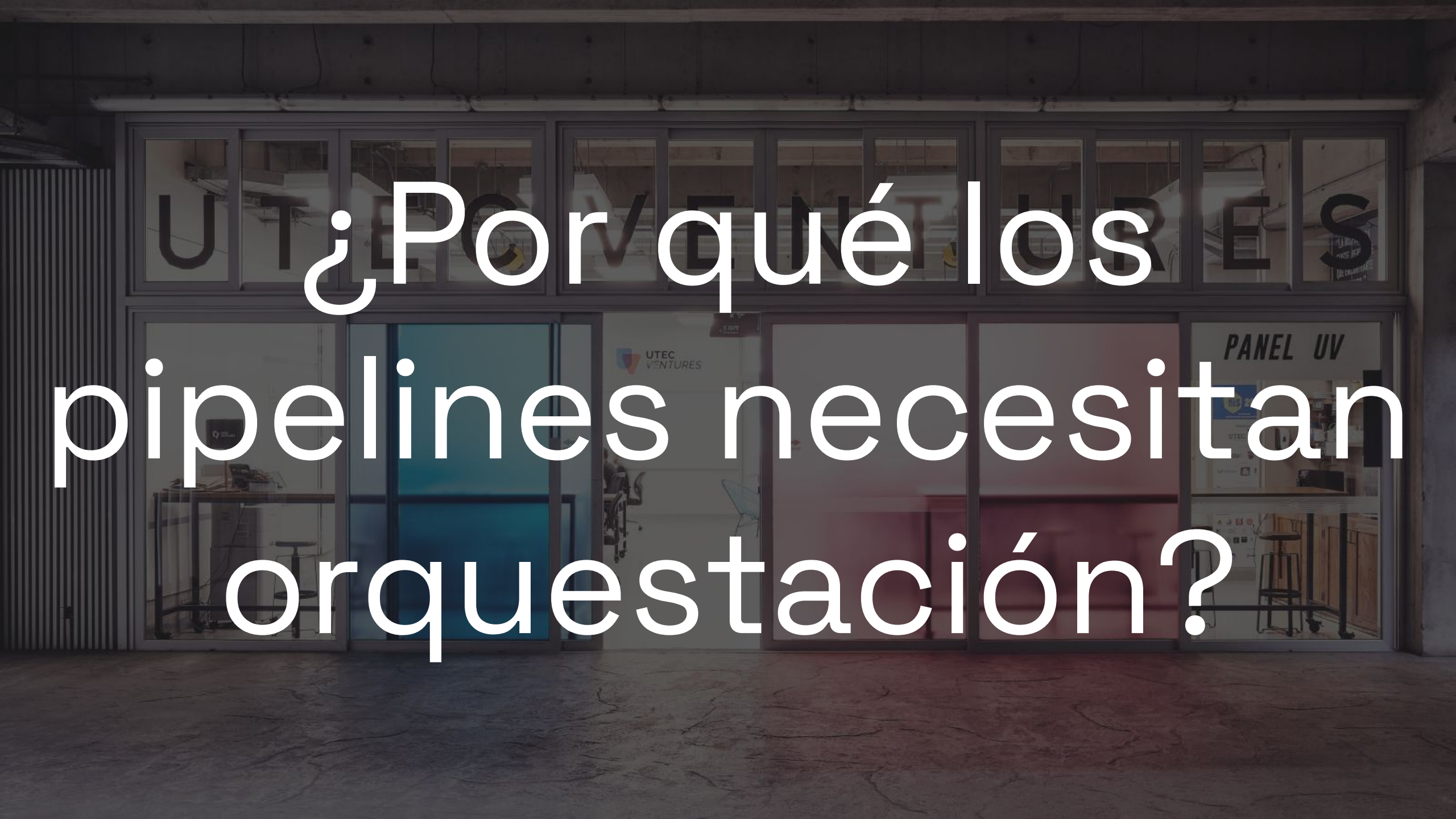
- Fundamentos de Data Engineering
- Diferencias entre Data Engineering, Data Discovery y Data Science
- Arquitecturas modernas de datos: Data Warehouses, Data Lakes y Lakehouse
- Data Mesh y Data Fabric: enfoques modernos para la gestión de datos
- Patrones de ingestión de datos: Batch, Streaming y Lambda/Kappa Architectures





- **Orquestación de flujos de datos.**
- Introducción a DataOps: CI/CD para ingeniería de datos.
- ETL (Extract, Transform, Load) y ELT: conceptos y diferencias.
- Implementación de flujos de integración de datos con herramientas modernas.



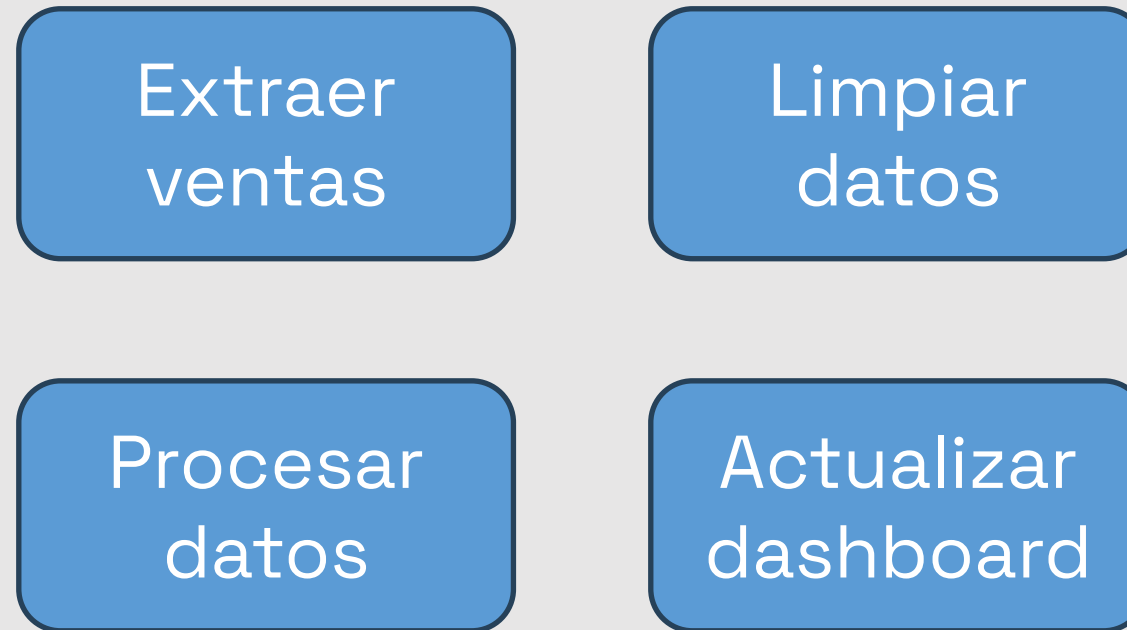


¿Por qué los
pipelines necesitan
orquestación?

¿Qué pasa si una tarea depende de otra... y se ejecutan fuera de orden?

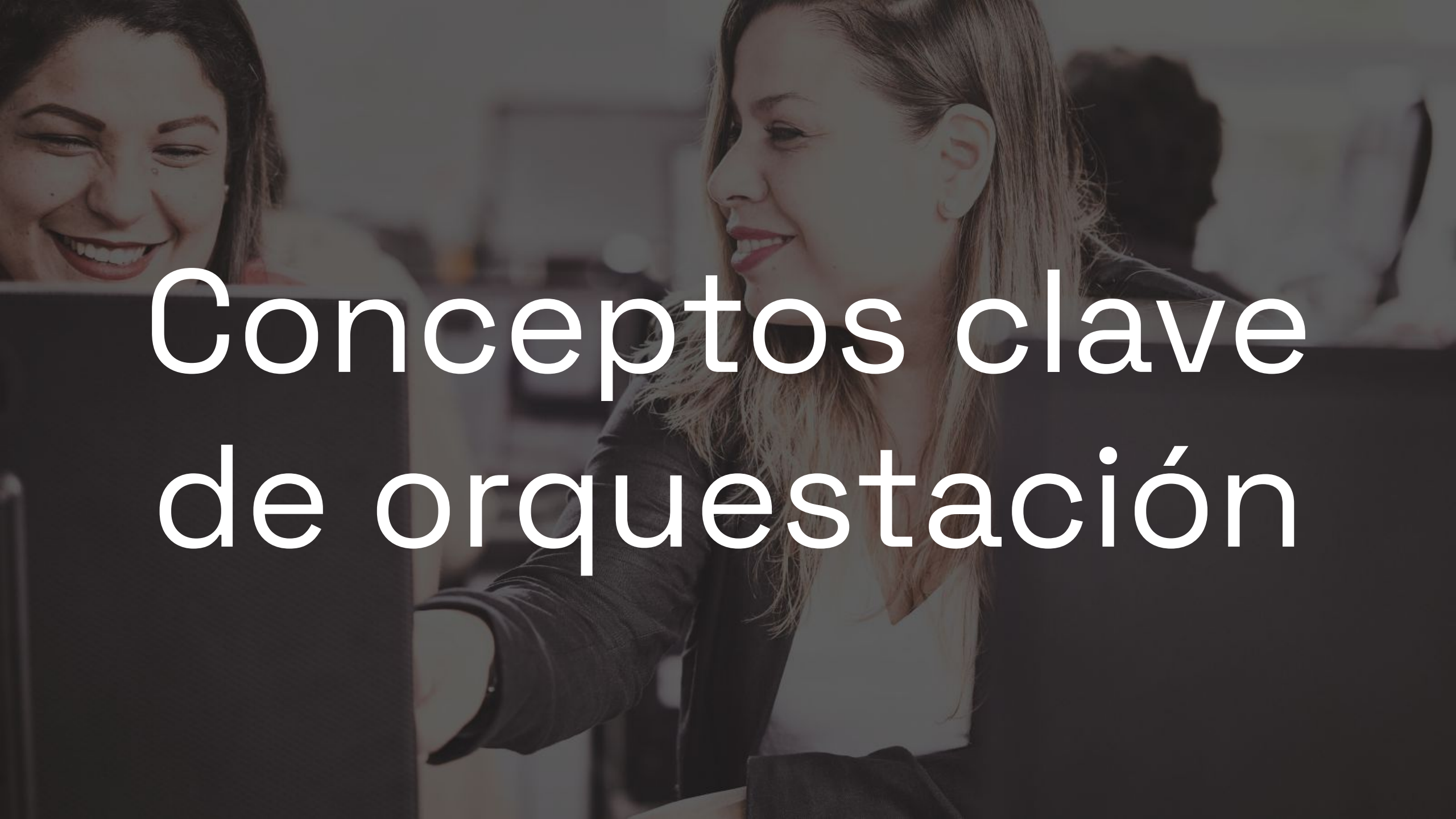


Ejemplo: Pipeline



¿Qué ocurre si el dashboard se actualiza antes de terminar la limpieza?



A background image showing two women in an office. The woman on the left is smiling broadly, looking down. The woman on the right is also smiling, looking towards the left. They appear to be in a collaborative work environment.

Conceptos clave de orquestación

¿Qué es Orquestación de datos?

“La orquestación de datos es el **control automatizado y programado de tareas interdependientes** que conforman un flujo de procesamiento, garantizando que se ejecuten en el orden, momento y condiciones correctas”



¿Qué hace realmente un orquestador?

- Ejecuta tareas automáticamente
- Respeta dependencias
- Maneja errores
- Reintenta tareas fallidas
- Centraliza monitoreo



A photograph of three people—two men and one woman—standing on a wide, modern concrete staircase. The man on the left is wearing a blue blazer over a light blue shirt and dark trousers. The woman in the center is wearing a light grey blazer over a mustard yellow top and dark trousers. The man on the right is wearing a light blue blazer over a light blue shirt and dark trousers. They are all smiling and looking towards the camera. The background shows the architectural details of the staircase and building, with a dark overlay and the text 'Apache Airflow' centered over the image.

Apache Airflow

¿Cómo coordinamos pipelines complejos usando código?



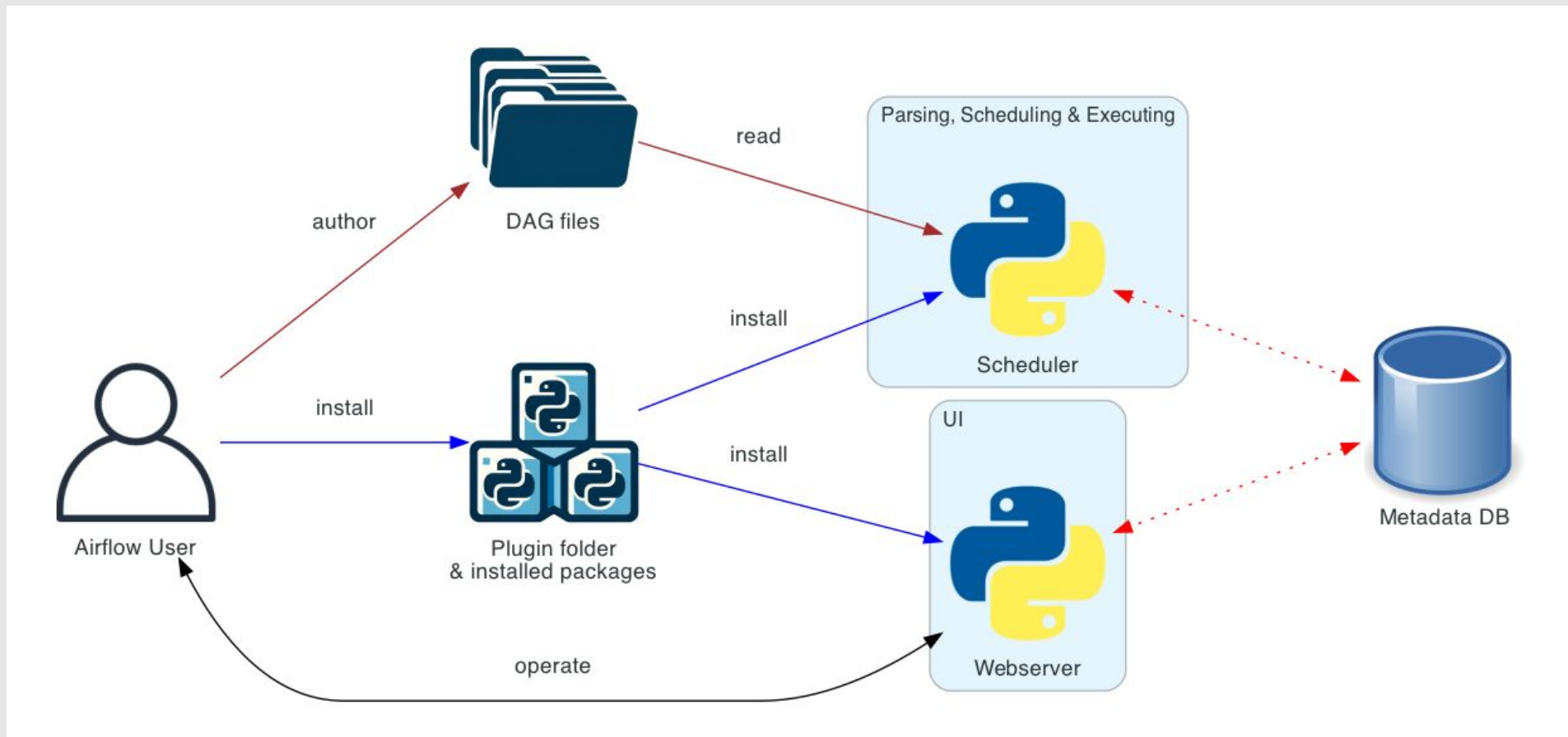
¿Qué es Apache Airflow?



Apache Airflow es una **plataforma de código abierto** para la **orquestación de flujos de trabajo programables**. Permite definir, programar y monitorizar flujos de trabajo como código (Workflows-as-Code), con una interfaz amigable para visualizar dependencias y controlar su ejecución.



Apache Airflow - Arquitectura



Apache Airflow – Componentes esenciales

Componente	Descripción
Scheduler	Planifica la ejecución de tareas según el DAG y las reglas de scheduling.
Procesador DAG	Lee y convierte los DAGs en estructuras que Airflow pueda usar.
Servidor Web	Interfaz web para visualizar, monitorear y gestionar los DAGs.
DAG Folder	Lugar donde se almacenan los DAGs (scripts .py).
Metadata Database	Base de datos que guarda el estado de los DAGs, tareas y logs.

Estructura de un DAG en Airflow

 dags/
└─ ejemplo_dag.py

 scripts/
└─ transformacion.py

 data/
└─ input.csv
└─ output.csv



```
@dag(
    dag_id='09_dag_etl_pro',
    start_date=datetime(2023, 1, 1),
    schedule="@daily",
    catchup=False,
    tags=["transformación", "ejemplo"]
)

def dag_etl_completo():

    @task
    def extraer():
        logging.info(f"==> Extrayendo datos")
        df = leer_datos(path=INPUT_FILE_PATH)
        return df.to_json() # pasar como string serializable

    @task
    def transformar(df_json):
        logging.info(f"==> Transformando datos")
        df = pd.read_json(df_json)
        df_transformado = transformar_datos(df)
        guardar_datos(df_transformado, TRANSFORMED_FILE_PATH)
        return df_transformado.to_json()

    @task
    def resumir(df_json):
        logging.info(f"==> Resumiendo datos")
        df = pd.read_json(df_json)
        resumen = resumir_datos(df)
        guardar_datos(resumen, SUMMARY_FILE_PATH)

    datos = extraer()
    datos_transformados = transformar(datos)
    resumir(datos_transformados)

dag = dag_etl_completo()
```



Anatomía de un DAG en Airflow

@dag

Decorator que indica a Airflow que la siguiente función representa un DAG

Atributo	Definición
dag_id	Nombre único del DAG dentro de Airflow.
start_date	Fecha desde la cual Airflow considera válido ejecutar el DAG.
schedule	Frecuencia de ejecución: @daily, @hourly, 0 8 * * *
catchup	Controla si Airflow ejecutará períodos antiguos pendientes.
tags	Etiquetas para organizar DAGs en la UI.

```
@dag(
    dag_id='09_dag_etl_pro',
    start_date=datetime(2023, 1, 1),
    schedule="@daily",
    catchup=False,
    tags=["transformación", "ejemplo"]
)

def dag_etl_completo():

    @task
    def extraer():
        logging.info(f"==> Extrayendo datos")
        df = leer_datos(path=INPUT_FILE_PATH)
        return df.to_json() # pasar como string serializable

    @task
    def transformar(df_json):
        logging.info(f"==> Transformando datos")
        df = pd.read_json(df_json)
        df_transformado = transformar_datos(df)
        guardar_datos(df_transformado, TRANSFORMED_FILE_PATH)
        return df_transformado.to_json()

    @task
    def resumir(df_json):
        logging.info(f"==> Resumiendo datos")
        df = pd.read_json(df_json)
        resumen = resumir_datos(df)
        guardar_datos(resumen, SUMMARY_FILE_PATH)

    datos = extraer()
    datos_transformados = transformar(datos)
    resumir(datos_transformados)

dag = dag_etl_completo()
```



Anatomía de un DAG en Airflow

@tag

Convierte una función Python en una tarea de Airflow.

Airflow:

- Los monitorea
- Genera logs
- Maneja retries
- Registra estados

```
@dag(  
    dag_id='09_dag_etl_pro',  
    start_date=datetime(2023, 1, 1),  
    schedule="@daily",  
    catchup=False,  
    tags=["transformación", "ejemplo"]  
)  
  
def dag_etl_completo():  
  
    @task  
    def extraer():  
        logging.info(f"==> Extrayendo datos")  
        df = leer_datos(path=INPUT_FILE_PATH)  
        return df.to_json() # pasar como string serializable  
  
    @task  
    def transformar(df_json):  
        logging.info(f"==> Transformando datos")  
        df = pd.read_json(df_json)  
        df_transformado = transformar_datos(df)  
        guardar_datos(df_transformado, TRANSFORMED_FILE_PATH)  
        return df_transformado.to_json()  
  
    @task  
    def resumir(df_json):  
        logging.info(f"==> Resumiendo datos")  
        df = pd.read_json(df_json)  
        resumen = resumir_datos(df)  
        guardar_datos(resumen, SUMMARY_FILE_PATH)  
  
    datos = extraer()  
    datos_transformados = transformar(datos)  
    resumir(datos_transformados)  
  
dag = dag_etl_completo()
```



Anatomía de un DAG en Airflow

Dependencia entre tareas
Airflow detecta automáticamente:

Extraer □ **Transformar** □ **Resumir**

Gracias al TaskFlow API las dependencias se infieren mediante llamadas entre funciones

```
@dag(
    dag_id='09_dag_etl_pro',
    start_date=datetime(2023, 1, 1),
    schedule="@daily",
    catchup=False,
    tags=["transformación", "ejemplo"]
)

def dag_etl_completo():

    @task
    def extraer():
        logging.info(f"==> Extrayendo datos")
        df = leer_datos(path=INPUT_FILE_PATH)
        return df.to_json() # pasar como string serializable

    @task
    def transformar(df_json):
        logging.info(f"==> Transformando datos")
        df = pd.read_json(df_json)
        df_transformado = transformar_datos(df)
        guardar_datos(df_transformado, TRANSFORMED_FILE_PATH)
        return df_transformado.to_json()

    @task
    def resumir(df_json):
        logging.info(f"==> Resumiendo datos")
        df = pd.read_json(df_json)
        resumen = resumir_datos(df)
        guardar_datos(resumen, SUMMARY_FILE_PATH)

    datos = extraer()
    datos_transformados = transformar(datos)
    resumir(datos_transformados)

dag = dag_etl_completo()
```



Anatomía de un DAG en Airflow

Instanciación final del DAG

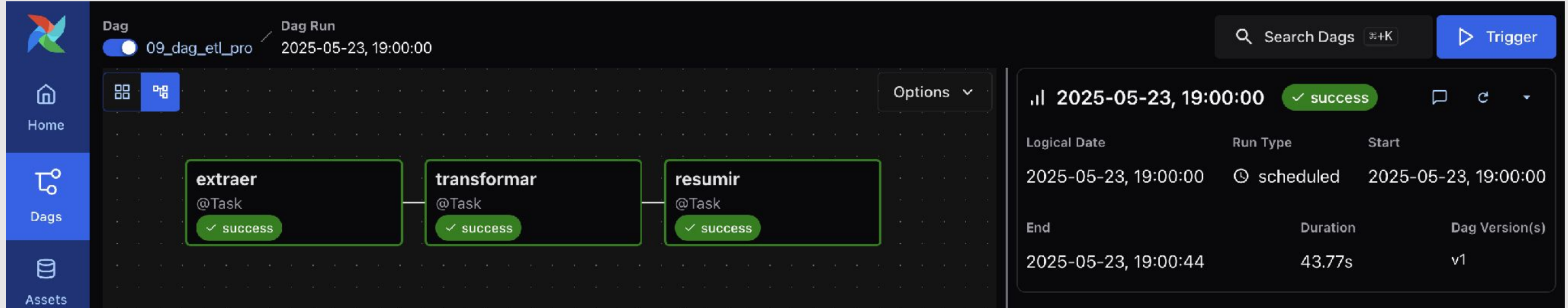
`dag = dag_etl_completo()`

Crea la instancia final del DAG
y permite que Airflow lo
descubra automáticamente

```
@dag(  
    dag_id='09_dag_etl_pro',  
    start_date=datetime(2023, 1, 1),  
    schedule="@daily",  
    catchup=False,  
    tags=["transformación", "ejemplo"]  
)  
def dag_etl_completo():  
  
    @task  
    def extraer():  
        logging.info(f"==> Extrayendo datos")  
        df = leer_datos(path=INPUT_FILE_PATH)  
        return df.to_json() # pasar como string serializable  
  
    @task  
    def transformar(df_json):  
        logging.info(f"==> Transformando datos")  
        df = pd.read_json(df_json)  
        df_transformado = transformar_datos(df)  
        guardar_datos(df_transformado, TRANSFORMED_FILE_PATH)  
        return df_transformado.to_json()  
  
    @task  
    def resumir(df_json):  
        logging.info(f"==> Resumiendo datos")  
        df = pd.read_json(df_json)  
        resumen = resumir_datos(df)  
        guardar_datos(resumen, SUMMARY_FILE_PATH)  
  
    datos = extraer()  
    datos_transformados = transformar(datos)  
    resumir(datos_transformados)  
  
    dag = dag_etl_completo()
```



Apache Airflow – Interfaz de Usuario



The screenshot displays the Apache Airflow web interface. On the left, a sidebar contains navigation links for Home, Dags, and Assets. The main area shows a DAG named '09_dag_etl_pro' with a 'Dag Run' status of '2025-05-23, 19:00:00'. The DAG consists of three tasks: 'extraer', 'transformar', and 'resumir', all of which are marked as 'success'. On the right, a table provides details for the selected DAG run.

Logical Date	Run Type	Start
2025-05-23, 19:00:00	⌚ scheduled	2025-05-23, 19:00:00
End	Duration	Dag Version(s)
2025-05-23, 19:00:44	43.77s	v1



UTEC VENTURES

Databricks Workflows

PANEL UV

UIS 20

UTEC

proyecto

¿Qué pasa si toda mi plataforma de datos ya vive en Databricks?



¿Qué es Databricks?



databricks

- Plataforma basada en **Apache Spark** para análisis y machine learning.
- Integra notebooks colaborativos, procesamiento batch y en streaming, manejo de datos, ML y más.
- Muy utilizada para trabajar con **grandes volúmenes de datos** en la nube.



¿Qué es un pipeline en Databricks?

- Serie de pasos conectados que procesan datos, como ETL o entrenamiento de modelos.
- Se pueden construir con:
 - Delta Live Tables (DLT)
 - Workflows (Jobs)
 - Pipelines en notebooks.



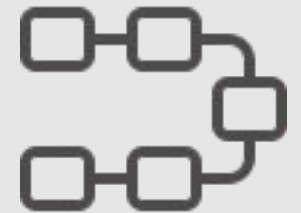
Notebooks en Databricks

- Entorno interactivo con soporte para Python, SQL, Scala, R.
- Integrado con control de versiones, visualizaciones, parámetros.
- Permite combinar código + documentación (como Jupyter, pero más robusto y colaborativo).

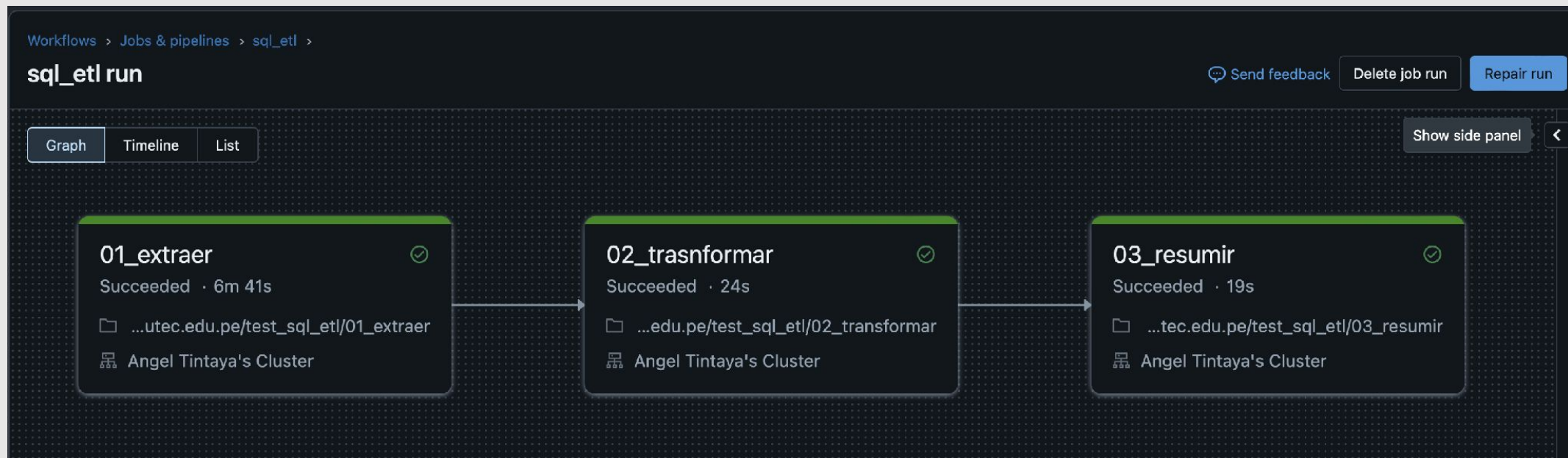


Workflows en Databricks

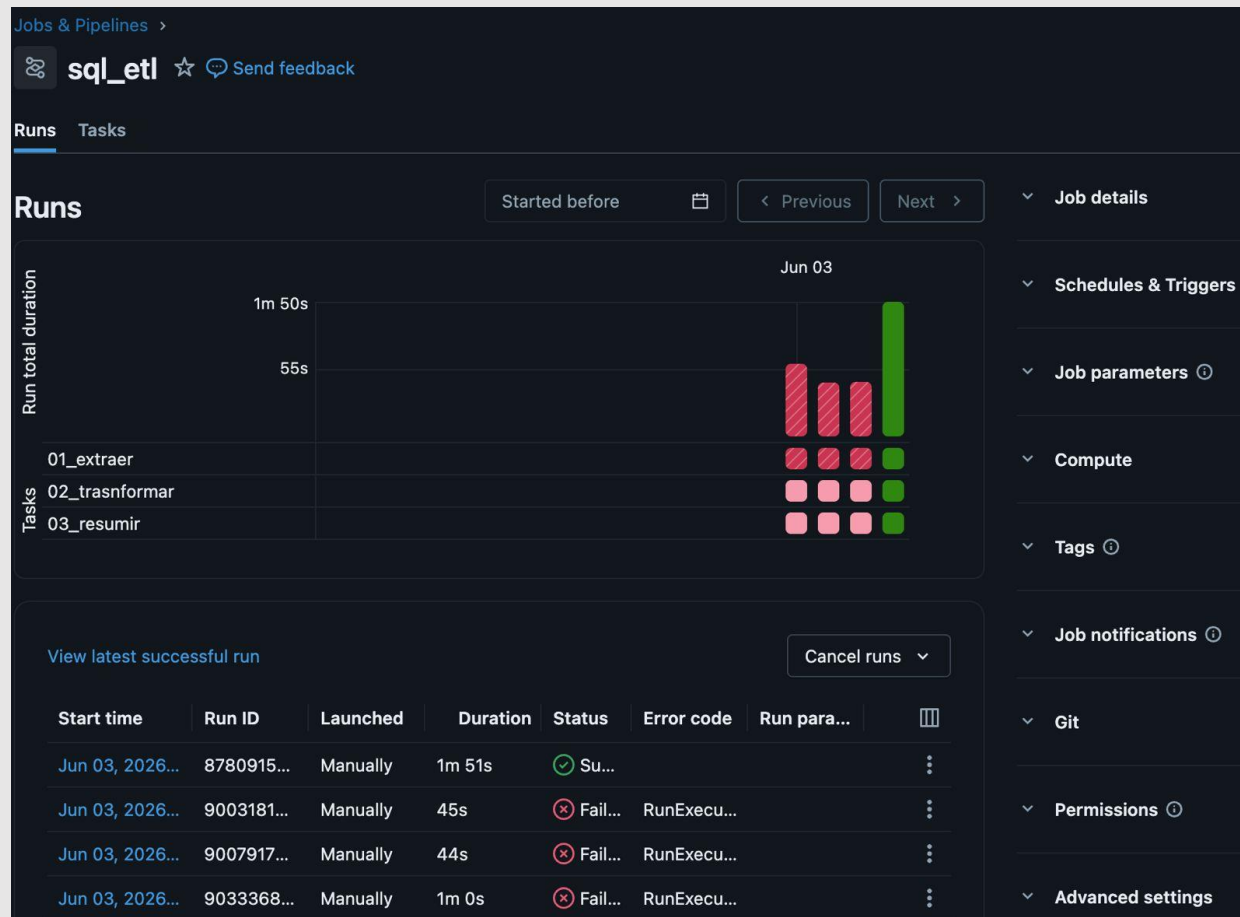
- Sistema de orquestación visual que permite ejecutar tareas de notebooks, Python scripts, etc.
- Ideal para procesos de ETL, entrenamiento de modelos o batch pipelines.



Workflow en Databricks



Monitoreo de Ejecuciones de un Job



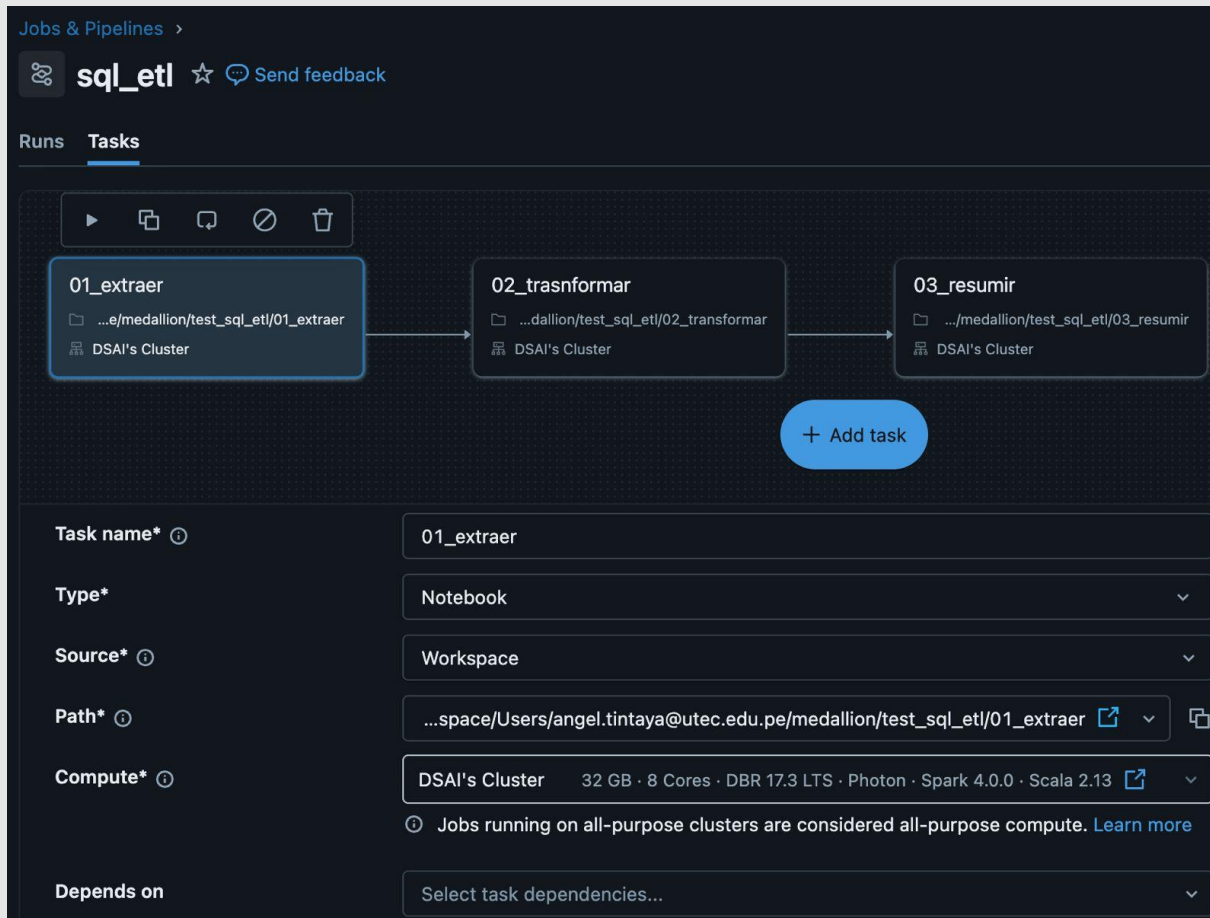
Monitoreo de Ejecuciones

Atributo	Definición
Start Time	Memento en que fue ejecutado el job
Run ID	ID que identifica a la ejecución del job en databricks
Launched	Tipo de ejecución realizada en el job (Manually, Scheduled, ...)
Duration	Duración de ejecución del job
Status	Estado en que terminó el Job
Error code	Error generado por el Job

Anatomía de un Task en Databricks

Jobs & Pipelines > **sql_etl** ☆ Send feedback

Runs **Tasks**



Task configuration for '01_extraer':

- Task name***: 01_extraer
- Type***: Notebook
- Source***: Workspace
- Path***: ...space/Users/angel.tintaya@utec.edu.pe/medallion/test_sql_etl/01_extraer
- Compute***: DSAI's Cluster (32 GB · 8 Cores · DBR 17.3 LTS · Photon · Spark 4.0.0 · Scala 2.13)
- Depends on**: Select task dependencies...

Task

Atributo	Definición
Task name	Nombre de la tarea
Type	Tipo de archivo usado en el task (Notebook, Python, Job, ...)
Source	Recurso de donde viene el archivo (Workspace, Git Provider)
Path	Ruta en donde se encuentra el archivo.
Compute	Cluster en donde se ejecuta el job
Depends on	Indica la dependencia del task.

Anatomía de un Task en Databricks

Jobs & Pipelines > **sql_etl** ☆ Send feedback

Runs **Tasks**

01_extraer → 02_trasnformar → 03_resumir

...e/medallion/test_sql_etl/01_extraer
DSAI's Cluster

...dallion/test_sql_etl/02_transformar
DSAI's Cluster

.../medallion/test_sql_etl/03_resumir
DSAI's Cluster

+ Add task

Parameters ⓘ UI JSON

Key	Value
TABLE_INPUT	g0_catalog.default.bronze_s {}

+ Add

Notifications ⓘ

✉ a.tintaya.m@gmail.com
On failure, On duration warning

Edit notifications

Retries ⓘ

5 sec delay, at most 1x (2 total attempts) ✎

Metric thresholds ⓘ

Duration warning: 5m 0s ✎

Task

Atributo	Definición
Parameters	Parametros usados en el job
Notifications	Mensaje enviado de acuerdo al evento generado por el Task
Retries	Número de intentos a realizar en caso el Task falle.
Metrics Thresholds	Métricas a considerar en las notificaciones: Tiempo esperado de ejecución, Tiempo máximo de ejecución.

Anatomía de un Job en Databricks

Job details

Job ID794420419956724
CreatorUTEC Angel
Run asUTEC Angel
DescriptionAdd description
Lineage2 upstream tables, 3 downstream tables
Performance optimizedOff

Schedules & Triggers

Paused - At 10:35 PM (UTC-05:00 — America/Bogota)

Edit triggerResumeDelete

Job parameters

No job parameters are defined for this job

Job	
Atributo	Definición
Job ID	ID que identifica al job en Databricks
Creator	Usuario que creó el Job
Run as	Usuario con el cual será ejecutado l job
Description	Descripción del job
Lineage	Linaje de datos del job
Performance Optimized	Reduce tiempo de setup del compute (> costo)
Schedulers	Programación de ejecucion

Anatomía de un Job en Databricks

Compute

Cluster running

DSAI's Cluster

Driver: Standard_D4ds_v4 · Workers: Standard_D4ds_v4 · 1-1 worker · Release: 17.3.13

Swap View details Spark UI Logs Metrics

Tags

team: G0

Job notifications

Git

Not configured

Add Git settings

Permissions

UTEC Angel Is Owner

admins Can Manage

Edit permissions

Job notifications

No notifications

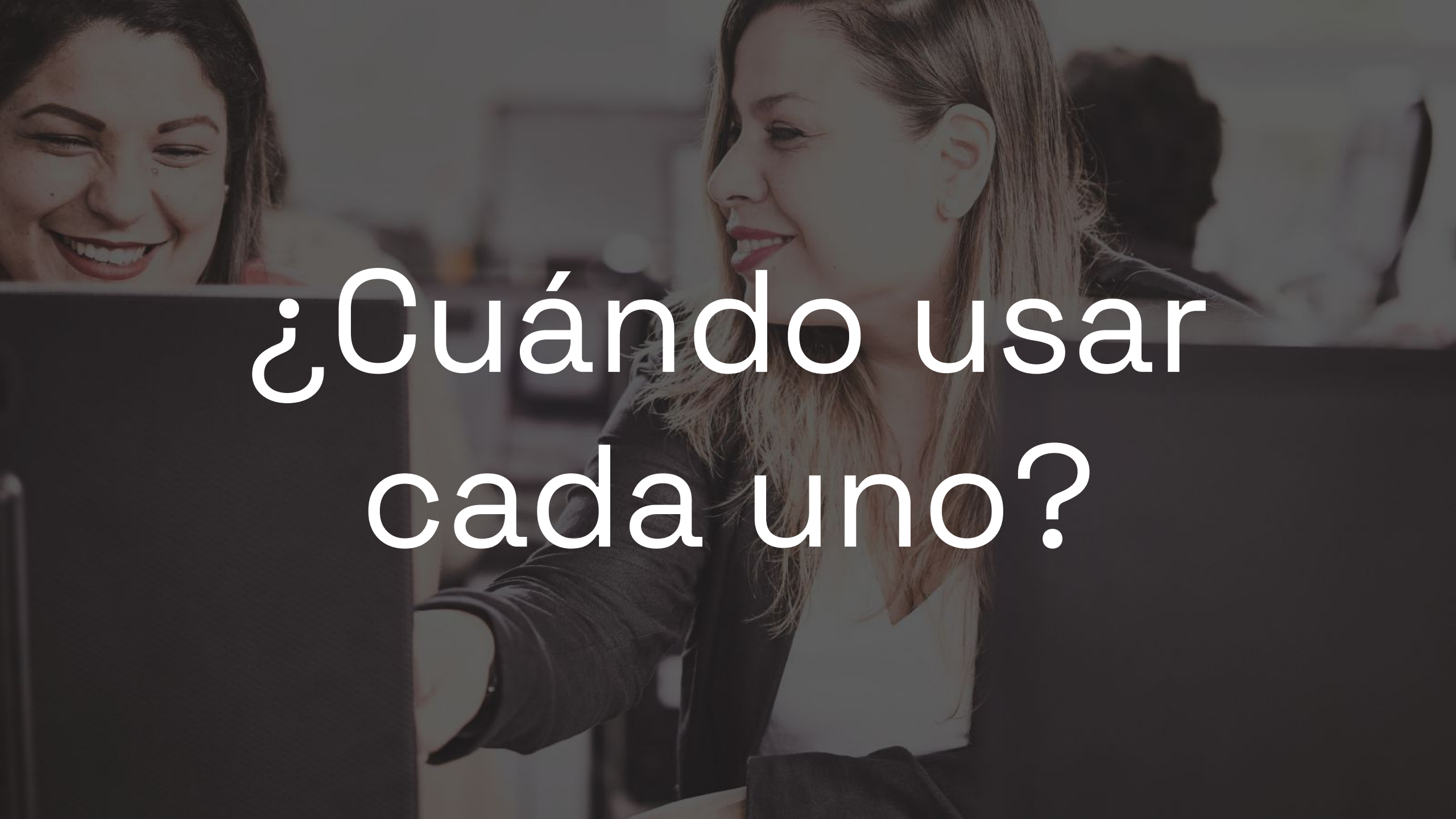
Edit notifications

Duration and streaming backlog thresholds

No thresholds defined

Add metric thresholds

Job	
Atributo	Definición
Compute	Cluster por el cual se ejecutará el job por defecto
Tags	Etiquetas para identificar Jobs en databricks
Job notifications	Mensaje enviado de acuerdo al evento generado por el Job
Git	Detalle del repositorio Git conectado al Job
Permissions	Usuarios que tienen permiso a editar el job

A background image showing two women in an office. The woman on the left is smiling broadly, and the woman on the right is also smiling and looking towards the left. They appear to be in a collaborative work environment.

¿Cuándo usar
cada uno?

Airflow vs Databricks Workflows

Criterio	Airflow	Databricks Workflows
Lenguaje principal	Python	Python (Notebooks)
Flexibilidad	Alta (cualquier operador)	Limitada al ecosistema Databricks
Casos ideales	Pipelines heterogéneos	Procesamiento Spark intensivo
Integraciones	Muchas (APIs, GCP, AWS, etc.)	Principalmente Spark, MLflow
Monitoreo	UI con logs detallados	Integrado a la plataforma
Complejidad inicial	Mayor curva de aprendizaje	Fácil para usuarios de Spark

Casos:

Caso 1: Banco Tradicional

Un banco tradicional tiene múltiples sistemas críticos:

- SAP on-premise
- APIs regulatorias
- SQL Server
- Snowflake
- Power BI
- Algunos procesos en Databricks

Cada noche deben ejecutarse decenas de procesos dependientes entre múltiples tecnologías.

Caso 2: Startup AI-first

Una startup de streaming musical construyó toda su plataforma sobre Databricks:

- Delta Lake
- Spark
- MLflow
- Streaming
- Notebooks
- Dashboards

El equipo quiere minimizar complejidad y operar con pocas herramientas.

Caso 3: E-commerce Global

Un e-commerce internacional opera con:

- APIs externas
- Kafka
- Azure SQL
- Databricks
- Modelos ML
- Dashboards ejecutivos

Necesitan coordinar integraciones corporativas y procesamiento analítico en Databricks.

Casos:

Caso 1: Banco Tradicional

Airflow

El entorno es heterogéneo y combina múltiples tecnologías (SAP, APIs, SQL Server, Snowflake y Databricks). Airflow permite coordinar dependencias complejas, centralizar monitoreo y desacoplar la orquestación de una sola plataforma.

Caso 2: Startup AI-first

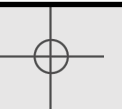
Databricks Workflows

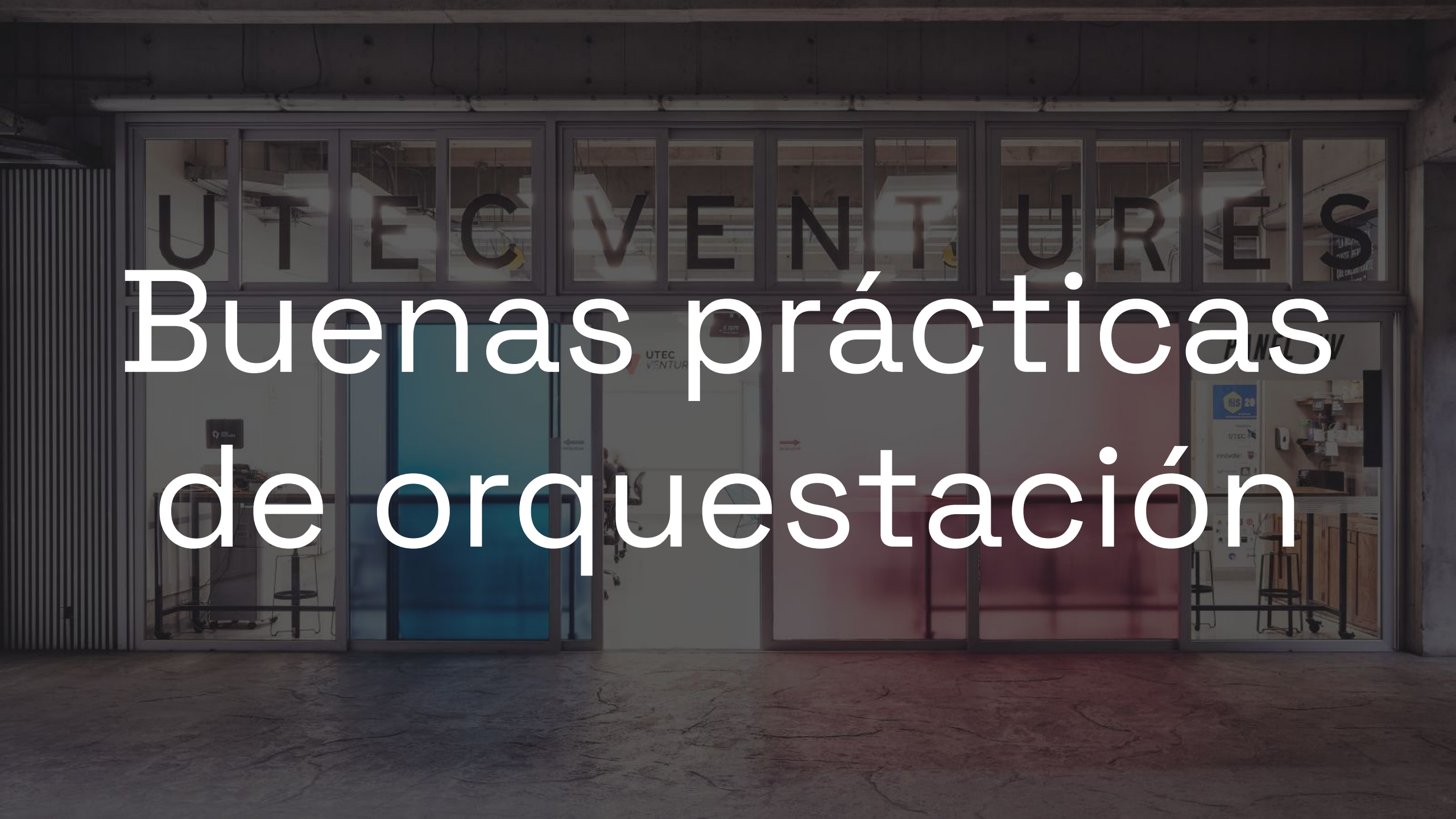
Toda la arquitectura ya vive dentro del ecosistema Databricks (Spark, Delta Lake, MLflow y streaming). Usar Workflows reduce complejidad operativa y facilita la integración natural entre pipelines, modelos ML y procesamiento distribuido.

Caso 3: E-commerce Global

Airflow + Databricks Workflows.

Airflow coordina integraciones corporativas y dependencias globales entre múltiples sistemas. Databricks Workflows ejecuta pipelines especializados de Spark, streaming y machine learning dentro del Lakehouse.





Buenas prácticas de orquestación

Buenas Prácticas de Orquestación

Buena práctica	¿Por qué importa?	Airflow	Databricks Workflows
Tareas pequeñas y reutilizables	Facilita mantenimiento y debugging	Tasks y DAGs modulares	Tasks separadas y notebooks desacoplados
Idempotencia	Evita duplicados o inconsistencias	Control en DAGs y lógica Python	Delta Lake + MERGE INTO
Retries automáticos	Recupera fallos temporales	retries y retry_delay	Retry policy en Jobs
Logging centralizado	Permite auditoría y troubleshooting	Logs por task y UI	Job logs y Spark UI

Buenas Prácticas de Orquestación

Buena práctica	¿Por qué importa?	Airflow	Databricks Workflows
Observabilidad	Monitoreo de pipelines y estados	DAG view y métricas	Job Runs y monitoring
Versionamiento	Control de cambios y rollback	DAGs en Git	Git folders y notebooks versionados
Alertas	Detectar fallos rápidamente	Email/Slack callbacks	Notifications por Job
Dependencias explícitas	Garantiza orden correcto	DAGs dirigidos	Dependencias visuales entre tasks

**¿Qué es más peligroso: que un pipeline falle...
o que falle silenciosamente?**





Takeaways

- La orquestación coordina pipelines complejos de forma confiable.
- Las dependencias y el monitoreo son críticos en sistemas productivos.
- Airflow destaca por su flexibilidad e integración heterogénea.
- Databricks Workflows simplifica pipelines dentro del ecosistema Databricks.
- Diseñar tareas pequeñas y observables mejora la mantenibilidad.
- La robustez operacional es tan importante como el procesamiento de datos.

¡Gracias!



UTEC Posgrado